

Vertex Memory Management Specification

Version: 1.0

Status: Draft

1. Objetivo

O Memory Management do Vertex é o subsistema responsável por controlar o uso de memória RAM por todo o sistema, incluindo Core, plugins e runtime de UI.

Seu objetivo é garantir estabilidade, evitar vazamentos de memória e otimizar o uso de recursos em dispositivos com restrições severas, como Wear OS.

Ele atua como camada central de controle de alocação, liberação e monitoramento de memória.

2. Responsabilidades

O Memory Management é responsável por:

- monitorar uso de memória em tempo real;
- controlar alocação e liberação de memória;
- impor limites por plugin;
- detectar vazamentos de memória;
- executar estratégias de coleta e liberação;
- cooperar com Resource Manager e Power Management;
- suspender ou encerrar processos sob pressão de memória;
- fornecer métricas de uso;
- otimizar cache interno do sistema.

3. Arquitetura

Vertex Core

↓

Memory Management Layer

- ├── Allocation Tracker
- ├── Garbage Strategy Controller
- ├── Memory Pool Manager
- └── Plugin Memory Quota Manager

- Leak Detector
- Pressure Monitor
- Cache Manager
- Cleanup Scheduler
- Memory Metrics

↓

JVM / ART Garbage Collector

↓

Native Memory (Android)

4. Componentes

Allocation Tracker

Registra todas as alocações feitas pelo sistema e plugins.

Garbage Strategy Controller

Coordena estratégias de liberação de memória.

Memory Pool Manager

Gerencia pools de memória reutilizável para reduzir overhead.

Plugin Memory Quota Manager

Define limites de memória por plugin.

Leak Detector

Detecta padrões de vazamento de memória.

Pressure Monitor

Monitora pressão geral de memória do sistema.

Cache Manager

Controla caches internos do Core.

Cleanup Scheduler

Agenda limpezas automáticas em momentos de baixa carga.

Memory Metrics

Coleta dados de uso e eficiência.

5. Fluxos

Alocação

Plugin / Core

↓

Request Memory

↓

Allocation Tracker

↓

Memory Pool / JVM

↓

Memory Granted

Pressão de memória

Pressure Monitor



Threshold Exceeded



Quota Manager



Cleanup Scheduler



Release Memory

Vazamento detectado

Leak Detector



Analysis



Report



Mitigation (cleanup / restart plugin)

6. APIs

Alocação

- allocate()
- free()
- reserve()

Monitoramento

- currentUsage()
- peakUsage()
- pressureLevel()

Controle

- setQuota()
- getQuota()
- enforceLimits()

Cache

- cache()
- clearCache()
- invalidateCache()

7. Estados

Sistema de memória

Normal

↓

Warning

↓

High Pressure

↓

Critical

↓

Recovery

Estado por plugin

- Healthy
- Limited
- Throttled
- Suspended
- Terminated

8. Políticas

Quotas por plugin

Cada plugin possui limite de memória definido dinamicamente.

Prioridade do Core

O Core tem prioridade absoluta sobre plugins em situações críticas.

Liberação agressiva

Sob pressão crítica, o sistema pode:

- liberar caches;
- suspender plugins;
- reiniciar componentes.

Detecção de vazamento

Memória suspeita de vazamento pode ser forçada a liberação.

Cooperação com GC

O sistema não substitui o GC da JVM/ART, mas o complementa.

Eficiência energética

Redução de uso de memória também reduz consumo de energia.

9. Integrações

O Memory Management integra-se com:

- Execution Runtime;
- Scheduler;
- Plugin Manager;
- Plugin Context;
- Resource Manager;
- Power Management;
- Threading Model;
- UI Runtime;
- State Engine;
- Event System;
- Logging Framework;
- Security Framework;
- Storage Engine;
- Cache System do Core.

10. Métricas

O sistema coleta:

- uso total de memória;
- uso por plugin;
- pico de consumo;
- frequência de pressão de memória;
- número de leaks detectados;
- número de limpezas executadas;
- cache hit/miss ratio;
- memória liberada por cleanup;
- tempo de resposta sob pressão.

11. Exemplos

Plugin consumindo memória excessiva

1. Plugin ultrapassa quota.

2. Memory Pressure é detectada.
3. Plugin entra em estado Throttled.
4. Cache é reduzido.
5. Se necessário, plugin é suspenso.

Situação crítica

1. Sistema entra em Critical Memory State.
2. Cleanup Scheduler é ativado.
3. Plugins não essenciais são suspensos.
4. Memória é liberada.
5. Sistema retorna ao estado Normal.

Leak detection

1. Plugin mantém referências crescentes.
2. Leak Detector identifica padrão.
3. Sistema registra alerta.
4. Plugin pode ser reiniciado ou isolado.

Uso normal

1. Plugins alocam memória normalmente.
2. Allocation Tracker registra uso.
3. Garbage Collector libera automaticamente.
4. Sistema permanece em estado Healthy.

12. Regras Arquiteturais

- Nenhum plugin pode alocar memória sem rastreamento.
- Memory quotas são obrigatórias.
- Core nunca pode ser suspenso por falta de memória.
- Liberação de memória deve ser segura e gradual sempre que possível.
- O sistema deve evitar GC thrashing.
- Vazamentos devem ser tratados como falhas de sistema.

13. Limitações

O Memory Management não é responsável por:

- lógica de execução de plugins;
- gerenciamento de UI;
- agendamento de tarefas;
- comunicação entre plugins;
- armazenamento persistente.

14. Futuras Extensões

O sistema poderá evoluir para:

- compressão de memória em tempo real;
- swapping inteligente entre RAM e storage;
- previsão de consumo de memória por plugin;
- isolamento por processo para plugins críticos;
- execução em memória compartimentada por segurança;
- integração com GPU memory management.

15. Resumo

O Memory Management do Vertex garante controle total sobre o uso de RAM em toda a plataforma. Ele monitora, limita e otimiza o consumo de memória de plugins e sistemas, prevenindo falhas críticas e mantendo o desempenho estável mesmo em dispositivos com recursos limitados. Em conjunto com Resource Manager e Power Management, forma o núcleo de estabilidade do sistema.